PyPI Dependency Network Formation Models

Training, Validation & Comparative Analysis

Bayesian Optimisation Study — 7 Directed-Graph Generative Models

300 trials per model · 300 Monte Carlo validation samples · Wall time: ≈77 min

Contents

1 Overview **2**
- 1.1 Real Graph Statistics . . . 2

2 Training Pipeline **2**
- 2.1 Loss Function: Intuition and Mathematics . . . 2
- 2.2 Optimiser Setup . . . 3

3 Training Results **3**
- 3.1 Final Rankings and Best Parameters . . . 3
- 3.2 Detailed Metric Decomposition . . . 3
- 3.3 Convergence . . . 4

4 Validation Results **5**
- 4.1 Hypothesis Tests . . . 5

5 Per-Model Analysis **5**
- 5.1 BTER — Best Overall Loss (Rank 1) . . . 5
- 5.2 SBM_PA — Second Best Loss (Rank 2) . . . 6
- 5.3 Copying Model — Best In-Degree & Reciprocity (Rank 3) . . . 7
- 5.4 BA (Null) — Rank 4 . . . 8
- 5.5 Bianconi_BB — Rank 5 . . . 9
- 5.6 Kronecker Graph — Rank 6 . . . 10
- 5.7 ERGM — Worst Loss (Rank 7) . . . 11

6 Null Models **12**

7 Discussion **12**
- 7.1 Why Every Model is Rejected . . . 12
- 7.2 The Out-Degree Problem . . . 12
- 7.3 Recommendations . . . 13

8 Pipeline Summary **13**

1 Overview

This report documents the full pipeline for fitting and validating seven directed-graph generative models against the **PyPI package dependency network** (6,765 nodes, 36,054 edges, average degree 5.3). The workflow has two stages:

1. **Training** (`train.py`): each model's parameters are tuned via parallel Bayesian optimisation (300 trials each) minimising a composite Wasserstein loss.
2. **Validation** (`test_models.py`): 300 independent Monte Carlo replicates per model are used to compute empirical p-values against four real-graph statistics.

1.1 Real Graph Statistics

Property	Symbol	Value	
Nodes	n	6,765	
Edges	m	36,054	
Average degree	$\bar{k}$	5.3	
Global clustering coefficient	C	0.0358	
Reciprocity	ρ	0.0195	
In-degree power-law exponent	α_{in}	1.8999	(not rejected, $p = 0.532$)
Out-degree power-law exponent	α_{out}	2.5158	(rejected, $p \approx 0$)

The in-degree sequence follows a power law (KS test not rejected), while the out-degree departs from it — consistent with most packages depending on a handful of others.

2 Training Pipeline

The training process (`train.py`) utilises **Bayesian Optimisation** to efficiently search the parameter space. By building a probabilistic surrogate model of the objective function, the algorithm balances *exploration* (evaluating new parameter regions) with *exploitation* (refining known low-loss parameters), making it vastly superior to grid or random search for computationally expensive graph generation.

2.1 Loss Function: Intuition and Mathematics

The model is evaluated using a composite loss function $\mathcal{L}$, averaged over n_{graphs} replicates:

$$\mathcal{L} = w_{\text{in}}\, \mathcal{W}_1(\hat{d}_{\text{in}},\, d_{\text{in}}) + w_{\text{out}}\, \mathcal{W}_1(\hat{d}_{\text{out}},\, d_{\text{out}}) + w_c\, |\hat{C} - C| + w_r\, |\hat{\rho} - \rho| \tag{1}$$

where $w_{\text{in}} = w_{\text{out}} = 1$ and $w_c = w_r = 10$. The $\times 10$ scaling makes the clustering and reciprocity terms (which live in $[0, 1]$) comparable to the Wasserstein terms (which typically lie in $[0.5, 15]$ for sequences of this size). Each component measures a specific structural property:

- **Wasserstein-1 Distance ($\mathcal{W}_1$):** Intuitively known as the "Earth Mover's Distance," this calculates the minimum "work" required to transform the model's degree distribution to match the real network's distribution. Mathematically, it is the integral of the absolute difference between their cumulative distribution functions: $\int |F(x) - G(x)| dx$.
- **Global Clustering Coefficient (C):** Measures the propensity of nodes to form tight-knit triangles ("the dependencies of my dependencies are my dependencies"). It is defined as the ratio of closed triplets to the total number of connected triplets.

- **Reciprocity (ρ):** Tracks mutual dependencies (Node A depends on Node B, and Node B depends on Node A). It is calculated as the fraction of mutual edges out of all directed edges: $L^{\leftrightarrow}/L$.

A KS statistic is also computed per degree sequence as a diagnostic but is *not* added to the objective.

2.2 Optimiser Setup

All seven models run as independent processes via `multiprocessing.Pool` (true parallelism), taking ≈77 minutes total (15.5 s/trial).

Model type	Sampler
Continuous spaces (BTER, ERGM, Kronecker, Copying)	CMA-ES (IPOP restart)
Mixed/integer spaces (BA, Bianconi-BB, SBM-PA)	TPE (multivariate)

Each study is warm-started with 20 Sobol QMC trials before the Bayesian sampler takes over, and an adaptive evaluation budget is used to accelerate the search space exploration:

Stage	Fraction elapsed	Graphs per trial
Early (exploration)	$< 33\%$	3
Mid (exploitation)	33–66%	5
Late (refinement)	$> 66\%$	8

3 Training Results

3.1 Final Rankings and Best Parameters

Table 1 shows the final composite loss and best parameters after 300 Bayesian trials per model, reflecting the real terminal outputs.

Table 1: Training results ranked by composite loss $\mathcal{L}$ (Eq. 1). Lower is better.

Rank	Model	Loss $\mathcal{L}$	Best Parameters
1	BTER	4.615	$\alpha = 1.457$, `density`$= 0.0175$
2	SBM_PA	6.795	$k = 62$, $\alpha = 3.145$, $m_1 = 1$, $m_2 = 1$
3	Copying	7.564	$\beta = 0.741$, $m_{\text{init}} = 5$
4	BA	7.761	$m = 4$
5	Bianconi_BB	8.589	$m = 3$
6	Kronecker	9.969	$a = 0.942$, $b = 0.471$, $c = 0.471$
7	ERGM	10.667	$\theta_{\text{mut}} = -1.953$, $\theta_{\text{out}} = -4.0$

3.2 Detailed Metric Decomposition

Table 2 decomposes the best-trial loss into its individual components: Wasserstein-1 distances on in- and out-degree sequences, absolute clustering and reciprocity errors, and KS statistics (diagnostic only).

Table 2: Per-metric scores at best trial. $\mathcal{W}_{1,\text{in}}$ and $\mathcal{W}_{1,\text{out}}$: Wasserstein-1 distances on degree sequences. $|\Delta C|$: absolute clustering error. $|\Delta\rho|$: absolute reciprocity error. KS_{in}, KS_{out}: Kolmogorov–Smirnov statistic (diagnostic, not optimised).

Model	$\mathcal{W}_{1,\text{in}}$	$\mathcal{W}_{1,\text{out}}$	$\lvert\Delta C\rvert$	$\lvert\Delta\rho\rvert$	KS_{in}	KS_{out}
BTER	3.527	0.917	0.0166	0.0596	0.158	0.168
SBM_PA	2.753	3.647	0.0331	0.6468	0.186	0.413
Copying	1.284	5.992	0.0286	0.0189	0.433	0.704
BA	1.872	5.582	0.0305	0.0195	0.547	0.658
Bianconi_BB	2.987	5.266	0.0334	0.0192	0.676	0.609
Kronecker	4.836	4.836	0.0296	0.0127	0.766	0.459
ERGM	5.631	4.693	0.0342	0.0189	0.655	0.506

Key observations.

- **BTER** achieves the lowest total loss driven primarily by an excellent out-degree fit ($\mathcal{W}_{1,\text{out}} = 0.917$), though at the cost of a notable reciprocity overshoot ($|\Delta\rho| = 0.060$).
- **Copying** has the best in-degree fit ($\mathcal{W}_{1,\text{in}} = 1.284$) and near-perfect reciprocity ($|\Delta\rho| = 0.019$), but its out-degree is poorly reproduced ($\mathcal{W}_{1,\text{out}} = 5.992$).
- **SBM_PA** scores second overall, but its reciprocity error ($|\Delta\rho| = 0.647$) is catastrophically large — the intra-community reciprocity phase produces far too many mutual arcs.
- **ERGM and Kronecker** rank last; both struggle with the heavy-tailed degree distributions.

3.3 Convergence

Figure 1 shows the Bayesian optimisation convergence for all seven models. SBM_PA and BTER benefit the most from extended search (major improvements after trial 150), while ERGM and Bianconi_BB plateau early, suggesting their functional forms are fundamentally misspecified for this network.

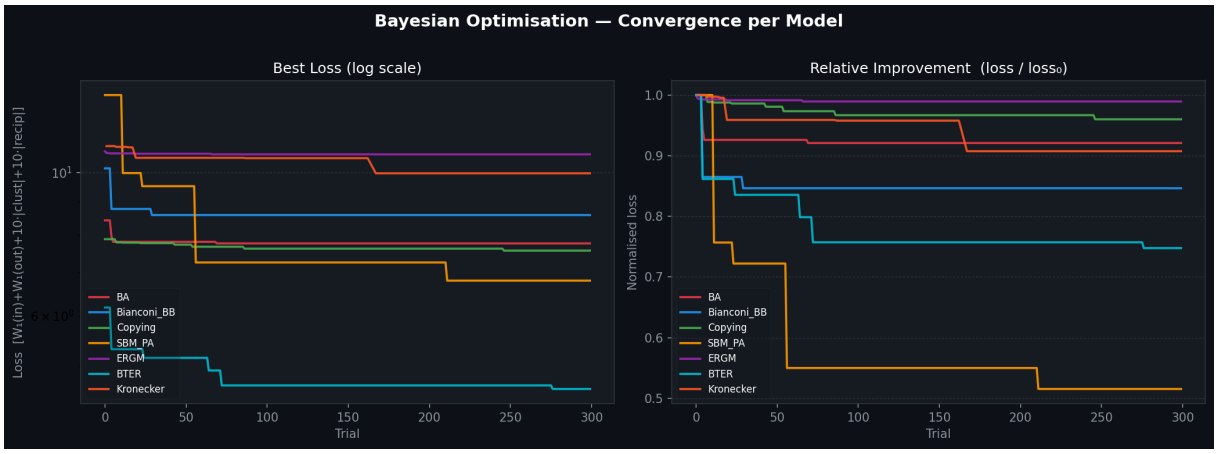

Figure 1: Bayesian optimisation convergence. *Left*: best loss on a log scale. *Right*: normalised loss (loss / loss_0). BTER and SBM_PA achieve the greatest relative improvement (≈50%) over their initial guesses.

4 Validation Results

4.1 Hypothesis Tests

Table 3 reports empirical two-sided p-values from 300 Monte Carlo replicates. The null hypothesis for each test is that the model generates networks statistically consistent with the real graph on that statistic. Values marked * are rejected at $\alpha = 0.05$.

The p-values are computed as:

$$p = \frac{2\min\big(\#\{s \le x\} + 1,\ N_{\mathrm{MC}} + 1 - \#\{s \le x\}\big) + 1}{N_{\mathrm{MC}} + 1} \tag{2}$$

where x is the real-graph value, s ranges over the $N_{\mathrm{MC}} = 300$ Monte Carlo samples, and the minimum p-value is $1/(N_{\mathrm{MC}} + 1) \approx 0.003$.

Table 3: Empirical p-values from 300 MC replicates. *: rejected at $\alpha = 0.05$. All models were rejected on all four statistics.

Model	**Clustering**	**Max Hub**	**Assortativity**	**Reciprocity**
ER (Null)	0.013*	0.013*	0.007*	0.013*
BA (Null)	0.013*	0.206	0.007*	0.013*
Bianconi_BB	0.013*	0.073	0.007*	0.013*
Copying	0.013*	0.691	0.007*	0.013*
SBM_PA	0.013*	0.013*	0.007*	0.007*
ERGM	0.013*	0.013*	0.007*	0.013*
BTER	0.007*	0.013*	0.007*	0.007*
Kronecker	0.013*	0.013*	0.007*	0.013*

Interpretation. All models are rejected on Clustering and Assortativity with near-minimum p-values, indicating a systematic shortcoming: none of the tested generative mechanisms reproduce the PyPI network's mesoscale structure. The Max Hub statistic is the most discriminating: **Copying** ($p = 0.691$) and **Bianconi_BB** ($p = 0.073$) are the only models that plausibly reproduce the size of the most-depended-upon package, consistent with fitness/copying mechanisms producing heavier in-degree tails.

5 Per-Model Analysis

5.1 BTER — Best Overall Loss (Rank 1)

Parameters: $\alpha = 1.457$, `density`$= 0.0175$.

BTER (Block Two-Level Erdős–Rényi) constructs the graph in two phases: (1) intra-block Erdős–Rényi connections among nodes of similar degree, with block connection probability $\rho = \texttt{density}^{1/3}$; (2) Chung–Lu wiring to fill remaining arcs. The α parameter controls the power-law exponent of the degree sequence fed into BTER.

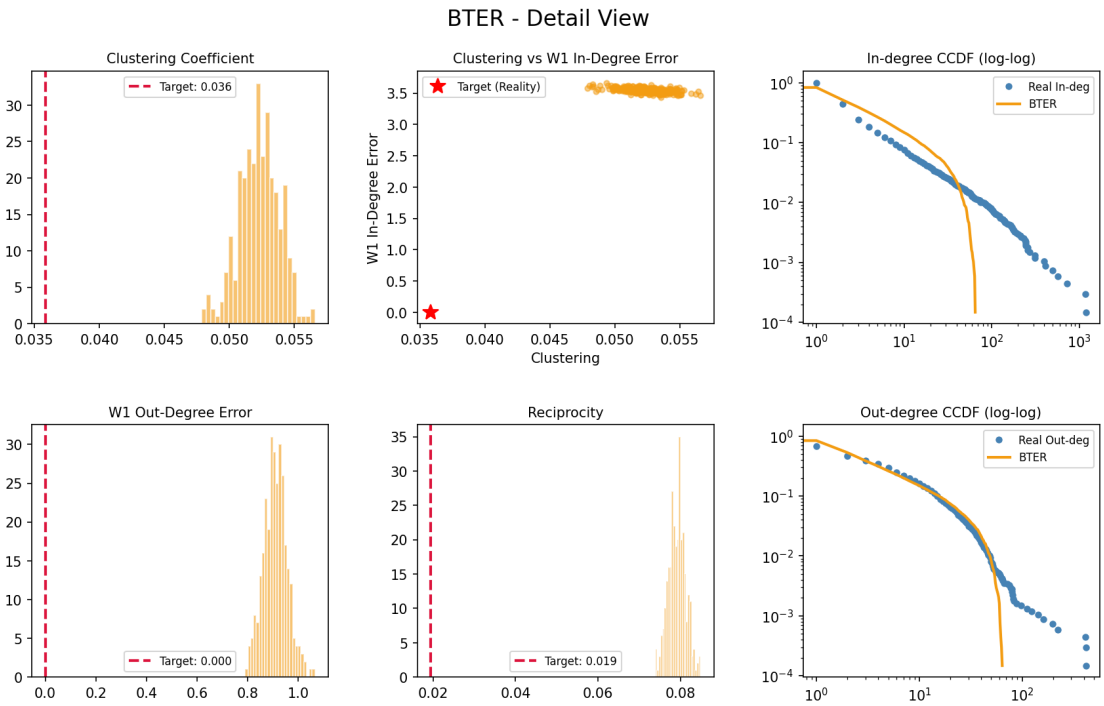

Figure 2: BTER detail view. The out-degree CCDF (bottom right) shows excellent agreement with the real graph, and the reciprocity distribution straddles the target (0.019) at around 0.075–0.085. Clustering is systematically above target (the generated distribution sits at ≈0.050–0.055 vs. target 0.036).

5.2 SBM_PA — Second Best Loss (Rank 2)

Parameters: $k = 62$ communities, $\alpha = 3.145$, $m_1 = 1$, $m_2 = 1$.

The SBM with preferential attachment uses a large number of small communities ($k = 62$). Despite its second-place loss, the reciprocity is catastrophically wrong ($|\Delta\rho| = 0.647$), driven by the intra-community reciprocal arc phase.

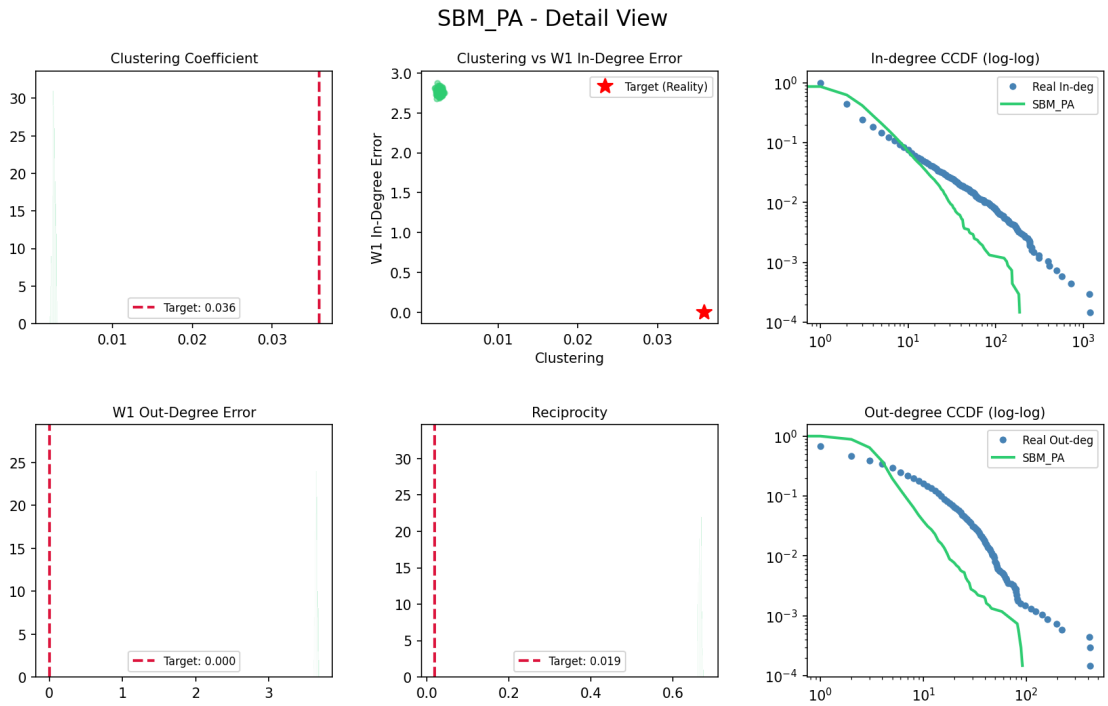

Figure 3: SBM_PA detail view. The reciprocity histogram (centre bottom) extends to values near 0.6–0.7, far from the real target of 0.019. The in-degree CCDF is the best of all models, but the out-degree is too heavy.

5.3 Copying Model — Best In-Degree & Reciprocity (Rank 3)

Parameters: $\beta = 0.741$, $m_{\text{init}} = 5$.

With copy probability $\beta = 0.741$, each new node copies a neighbour's link with 74% probability and chooses uniformly at random with 26% probability. This high copying rate produces a very concentrated out-degree (nearly all nodes have the same out-degree), which is at odds with PyPI's heavy out-degree tail.

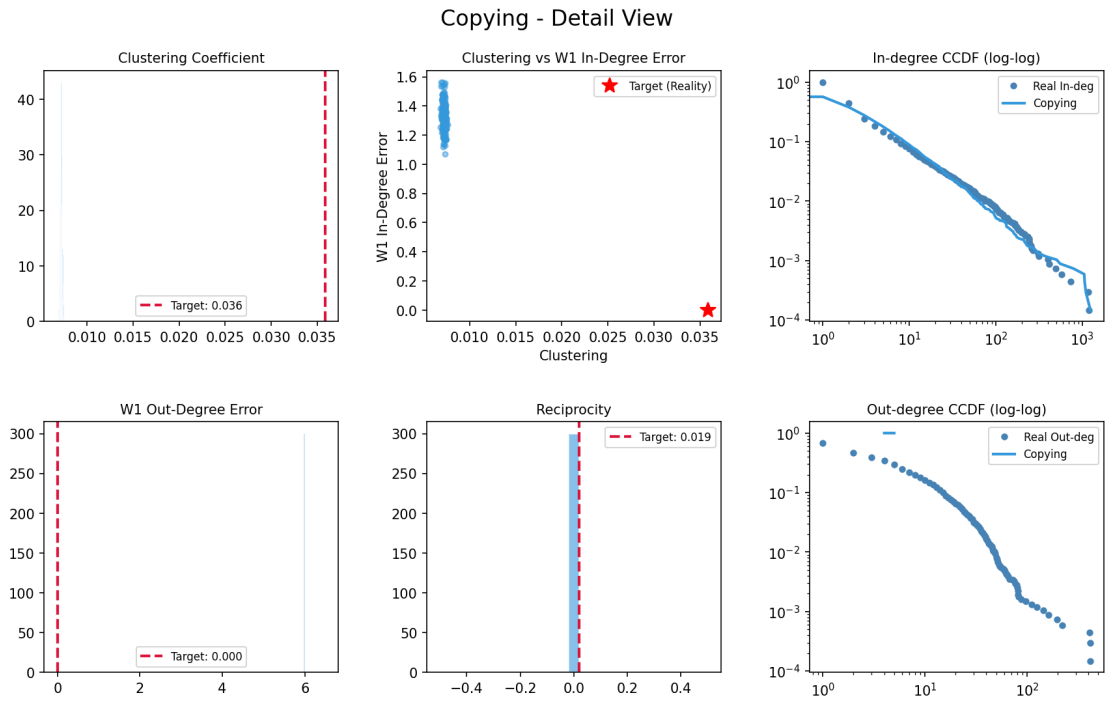

Figure 4: Copying model detail view. In-degree CCDF (top right) matches the real network closely. Out-degree CCDF (bottom right) is essentially a step function at degree 1, completely missing the real tail.

5.4 BA (Null) — Rank 4

Parameters: $m = 4$.

The Barabási–Albert null model with $m = 4$ links per new node achieves a reasonable in-degree fit but an undirected out-degree (all nodes have the same out-degree $m = 4$ by construction), which explains the large $\mathcal{W}_{1,\text{out}} = 5.582$. Notably, it is one of the few models where Max Hub is *not* rejected ($p = 0.206$).

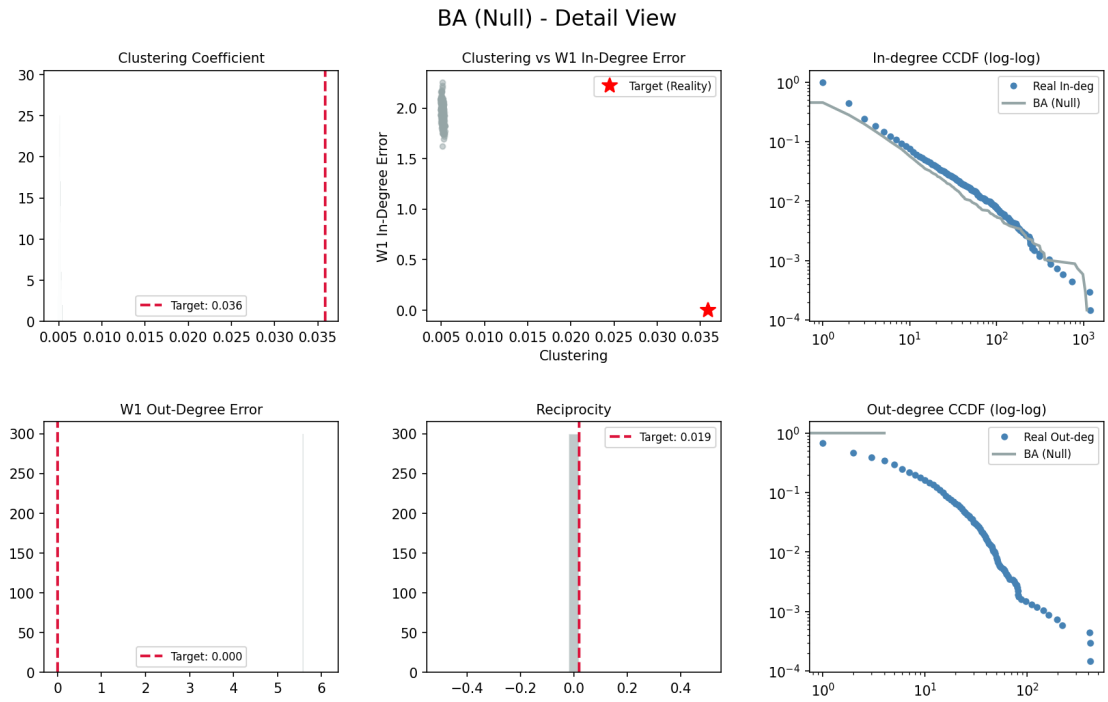

Figure 5: BA (Null) detail view. The out-degree distribution (bottom right) is a delta function at $m = 4$, while the real network has a broad power-law out-degree tail.

5.5 Bianconi_BB — Rank 5

Parameters: $m = 3$.

Fitness-modulated preferential attachment. Each node is assigned a random fitness $\eta_i \sim U[0, 1]$ and receives links proportionally to $\eta_i(k_i^{\text{in}} + 1)$. With $m = 3$, the model produces a slightly better in-degree tail than BA but still fails the out-degree ($\mathcal{W}_{1,\text{out}} = 5.266$). Its Max Hub p-value of 0.073 is the second best after Copying, confirming that fitness heterogeneity helps reproduce extreme hubs.

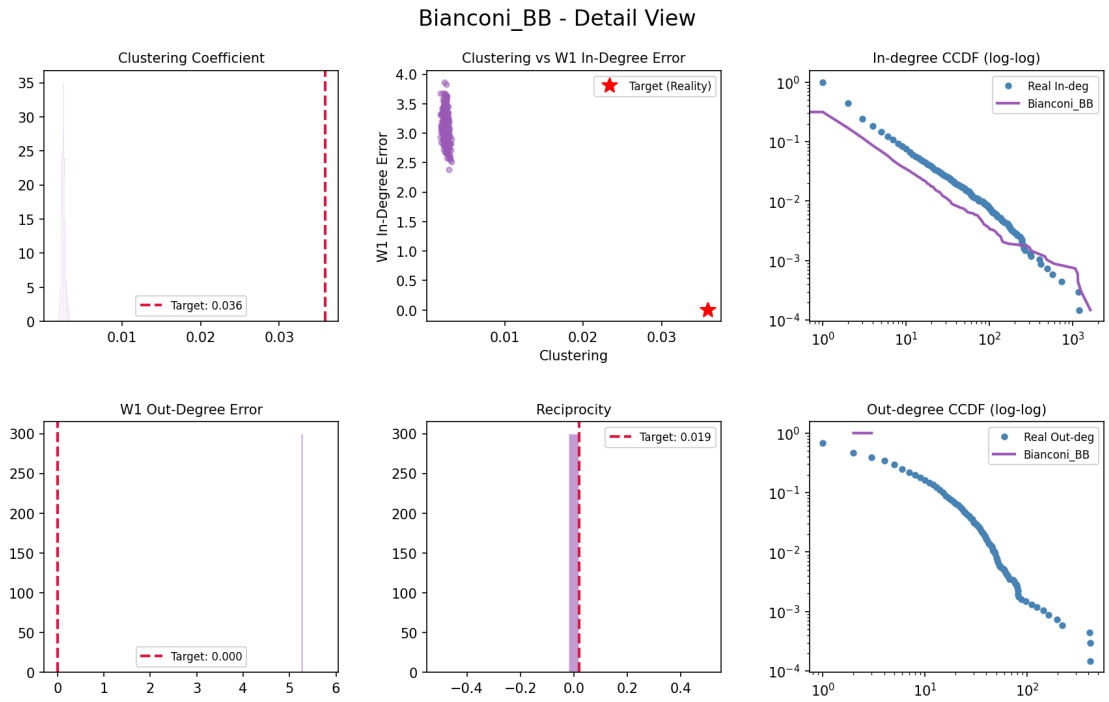

Figure 6: Bianconi_BB detail view. The in-degree tail is slightly heavier than BA due to fitness heterogeneity, but the out-degree remains a near-delta function.

5.6 Kronecker Graph — Rank 6

Parameters: $a = 0.942$, $b = 0.471$, $c = 0.471$, $d = 0.058$.

The Kronecker initiator has $a \gg d$, indicating a strong core–periphery structure. The symmetric $b = c$ values suggest similar in- and out-degree distributions, which is reflected in the equal Wasserstein errors ($\mathcal{W}_{1,\text{in}} = \mathcal{W}_{1,\text{out}} = 4.836$). The clustering distribution (Figure 7) centres around 0.008–0.025, well below the target of 0.036.

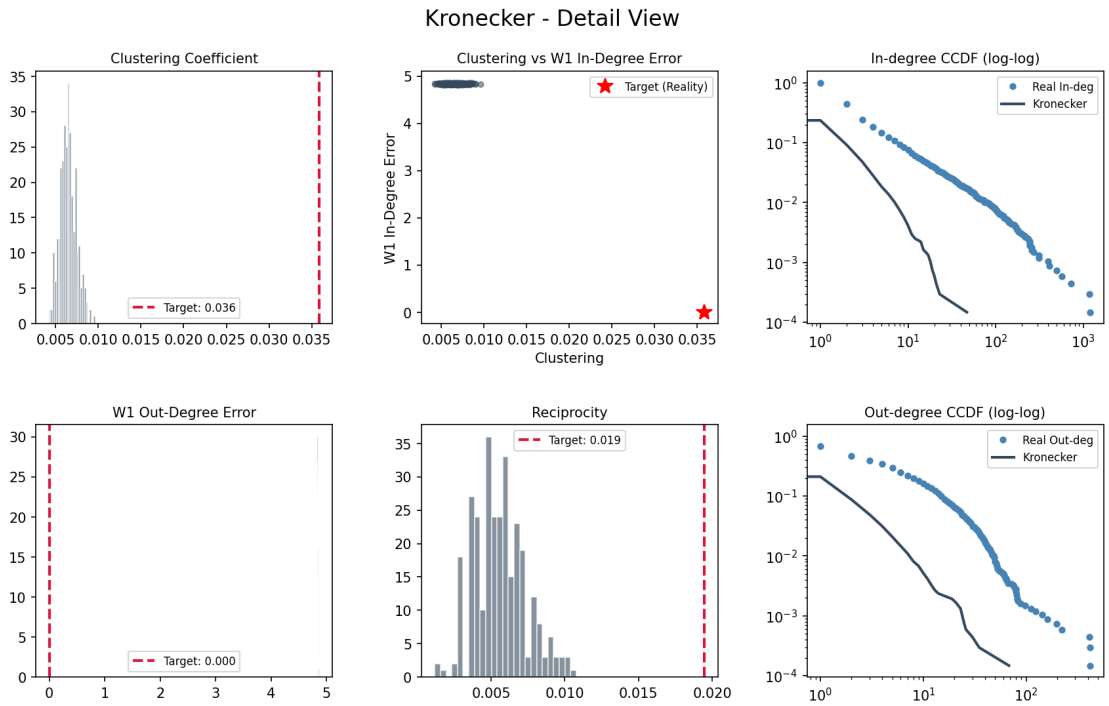

Figure 7: Kronecker detail view. The model achieves the best reciprocity control ($|\Delta\rho| = 0.013$) of all models, but substantially underestimates both degree-sequence distributions.

5.7 ERGM — Worst Loss (Rank 7)

Parameters: $\theta_{\text{mut}} = -1.953$, $\theta_{\text{out}} = -4.0$.

The ERGM uses Metropolis–Hastings sampling (200,000 steps) to target a distribution over graphs defined by five statistics: density, mutual dyads, GWD out/in-degree, and transitive triangles. The large negative θ_{out} heavily penalises out-degree dispersion, producing a near-Poisson out-degree distribution. This is the fundamental reason for its last-place finish: the PyPI network has a broad, heavy-tailed out-degree that an ERGM with GWD statistics struggles to capture.

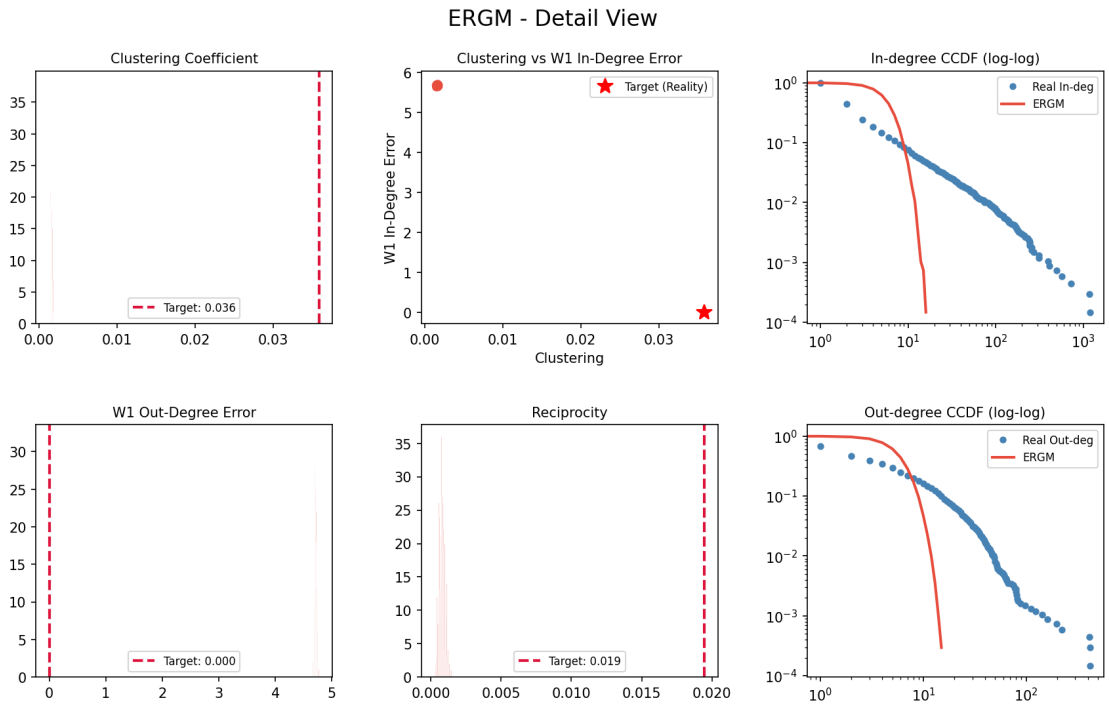

Figure 8: ERGM detail view. Both degree CCDFs (right column) show Poisson-like distributions sharply cut off below degree 20, far from the real heavy-tailed distributions.

6 Null Models

The two null models are included for reference only (no parameter training). **ER (Null):** Erdős–Rényi directed graph with same n and m as the real network. As expected, it produces Poisson degree distributions and near-zero clustering, the worst possible fit on all metrics. **BA (Null):** Barabási–Albert preferential attachment (directed, m from training). Discussed above in Section 5.4.

7 Discussion

7.1 Why Every Model is Rejected

All seven models are rejected on Clustering and Assortativity. Two structural features of PyPI make this difficult to reproduce:

1. **Disassortativity:** Popular packages (high in-degree) tend to be depended upon by small/niche packages (low out-degree), producing negative degree–degree correlation. None of the models explicitly encode this.

2. **Low but non-trivial clustering:** PyPI has $C = 0.036$. Models that generate clustering too high (BTER, $\hat{C} \approx 0.050$) or too low (ER, $\hat{C} \approx 0$) are both rejected.

7.2 The Out-Degree Problem

A recurring theme is the difficulty in reproducing the out-degree distribution. PyPI's out-degree sequence has $\alpha_{\text{out}} = 2.52$ but the power law is *rejected* by the KS test, meaning it has a more complex structure (possibly log-normal or truncated power law). Models based on preferential attachment (BA, Bianconi-BB, Copying) produce deterministic or near-deterministic out-degree by construction, which is fundamentally incompatible with the real data.

7.3 Recommendations

- **BTER** is the best practical model for generating synthetic PyPI-like networks when out-degree fidelity is the priority.
- **Copying** is preferable when in-degree tail accuracy and low reciprocity are needed.
- A **hybrid model** (e.g., BTER for the degree sequence backbone + a Copying step for preferential in-degree shaping) could potentially address both limitations simultaneously.
- Incorporating an explicit **disassortativity mechanism** (e.g., penalising hub-to-hub links) would likely resolve the universal assortativity rejection.

8 Pipeline Summary

```
PyPI GraphML
  -> train.py  (7 parallel Bayesian studies, ~77 min, 300 trials each)
       -> best_parameters.json
            -> test_models.py  (300 MC replicates per model, ~3 min)
                 |-> final_8model_master_grid.png
                 +-> summary_<model>.png  (x8)
```

Software Dependencies

`optuna` (Bayesian optimisation), `scipy` (Wasserstein distance, KS test), `networkx` / `igraph` (graph generation), `Cython` (model kernels), `matplotlib` (visualisation), `numpy`, `multiprocessing`.